

Sistema de Gerenciamento de Biblioteca

API + Painel Web

Curso Completo de JavaScript com Node.js

Node.js • Express • MVC • MySQL • Sequelize • EJS • REST API • JWT

Material didático — Apostila do Aluno

10 Aulas | Teoria + Prática Guiada + Desafios

Sobre este Material

Esta apostila acompanha o curso prático de JavaScript com Node.js, cujo objetivo é construir, aula a aula, um projeto completo e realista: o Sistema de Gerenciamento de Biblioteca (API + Painel Web).

O projeto foi desenhado para reunir exatamente os conceitos e tecnologias mais exigidos no mercado de desenvolvimento backend com JavaScript:

Autenticação de usuários (Login com JWT)

CRUD completo (Create, Read, Update, Delete)

Banco de Dados relacional (MySQL)

Arquitetura MVC (Model-View-Controller)

Framework Express.js

Template Engine EJS

API REST

Middlewares

Deploy (opcional)

Cada aula está estruturada em três partes: Teoria (conceitos explicados de forma direta, com analogias quando útil), Prática Guiada (código comentado, passo a passo, sempre construindo em cima da aula anterior) e Desafio (um exercício para o aluno aplicar sozinho o que aprendeu).



Como usar esta apostila

Recomenda-se seguir as aulas em ordem, sempre com o código da aula anterior já funcionando, pois cada aula evolui diretamente o mesmo projeto (biblioteca-node). Os desafios não são opcionais — resolvê-los é o que consolida o aprendizado antes de avançar.

Sumário

01

Aula 01 — Ambiente de Desenvolvimento e Node.js

03

Aula 03 — MVC (Model-View-Controller)

05

Aula 05 — ORM (Sequelize)

07

Aula 07 — EJS (Views Dinâmicas)

09

Aula 09 — Integração Front-end + Back-end

02

Aula 02 — Express.js

04

Aula 04 — Banco de Dados MySQL

06

Aula 06 — Async / Await

08

Aula 08 — REST API Completa

10

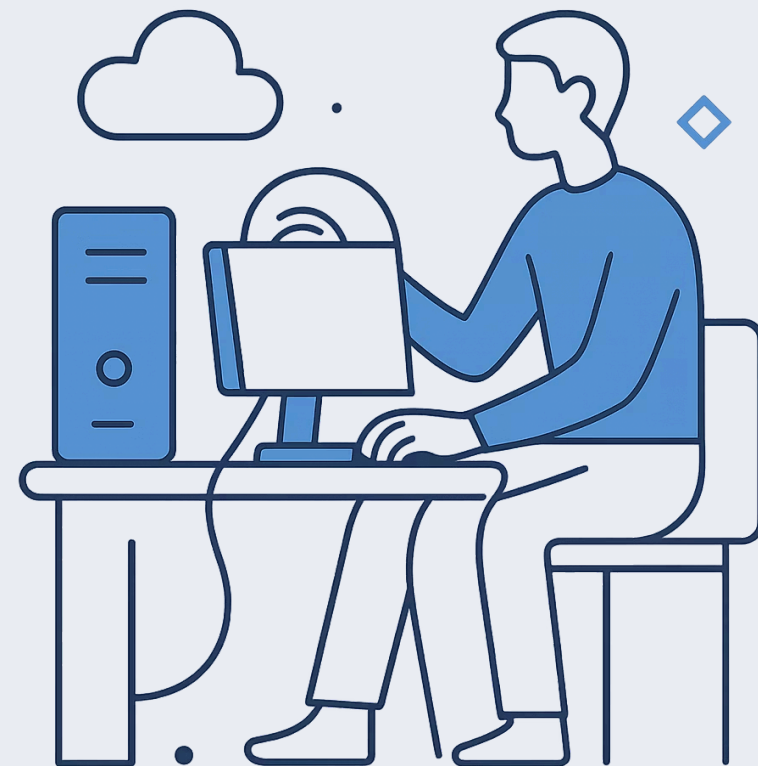
Aula 10 — Projeto Final: Sistema de Biblioteca Completo

Aula 01 — Ambiente de Desenvolvimento e Node.js

Duração total: 3 horas (30min teoria + 2h30 prática guiada).

Objetivos

Configurar todo o ambiente de desenvolvimento e compreender, na prática, como o Node.js funciona por trás dos panos — o que ele é, por que ele existe, e como ele executa JavaScript fora do navegador.



Teoria — O que é Backend?

Toda aplicação web é dividida em duas grandes partes:

Frontend

o que o usuário vê e interage — HTML, CSS, JavaScript rodando no navegador.

Backend

a parte que roda em um servidor, escondida do usuário. É responsável por regras de negócio, acesso a banco de dados, autenticação, envio de e-mails, geração de arquivos, integrações com outras APIs, etc.

No nosso projeto (Sistema de Gerenciamento de Biblioteca), o backend será responsável por: cadastrar livros e usuários, controlar empréstimos, validar login, e expor os dados via API para quem quiser consumir (painel web, app mobile, etc.).

Cliente x Servidor

O modelo cliente-servidor é a base da internet:

Cliente

quem faz a requisição (navegador, aplicativo mobile, Postman, outro servidor).

Servidor

quem recebe a requisição, processa e devolve uma resposta.

Esse ciclo é chamado de Request/Response (Requisição/Resposta). O cliente envia uma requisição HTTP (ex: 'me dá a lista de livros'), o servidor processa (consulta o banco, aplica regras) e devolve uma resposta (geralmente em JSON ou HTML).



Como funciona o Node.js

Runtime JavaScript

Um 'runtime' é o ambiente que fornece tudo que é necessário para o código rodar: o motor de execução (V8), APIs adicionais (como o módulo 'fs' para arquivos, ou 'http' para servidores) e um sistema de gerenciamento de memória. O Node.js adiciona essas capacidades ao JavaScript puro.

JavaScript nasceu para rodar dentro do navegador. O Node.js é um ambiente de execução (runtime) que permite rodar JavaScript fora do navegador, diretamente no seu computador ou em um servidor. Ele é construído sobre o V8, o mesmo motor que executa JavaScript no Google Chrome.

Isso significa que, com Node.js, podemos usar JavaScript para criar servidores, acessar arquivos do sistema operacional, conectar a bancos de dados, criar APIs — tarefas que antes só eram possíveis com linguagens como PHP, Java ou Python.

Event Loop — o coração do Node.js

Node.js é single-threaded (executa em uma única linha de execução principal), mas consegue lidar com milhares de requisições simultâneas graças ao Event Loop e à sua natureza assíncrona e não-bloqueante.

Na prática: quando o Node.js precisa fazer algo demorado (ler um arquivo, consultar um banco de dados, esperar uma resposta de rede), ele NÃO fica parado esperando. Ele delega essa tarefa e continua executando outras linhas de código. Quando a tarefa demorada termina, uma função de callback é chamada para tratar o resultado.

📋 💡 **Analogia do Garçom:** Imagine um garçom (Node.js) que atende várias mesas sozinho. Ele anota o pedido da mesa 1 e, em vez de ficar parado esperando a cozinha preparar o prato, vai atender a mesa 2, depois a mesa 3. Quando a cozinha avisa que o prato da mesa 1 ficou pronto (callback), ele entrega. Isso é o Event Loop: não bloquear a espera, e reagir quando o resultado chega.

Esse modelo é o motivo pelo qual o Node.js é excelente para APIs, sistemas que fazem muitas operações de I/O (entrada/saída) como acesso a banco de dados — exatamente o nosso caso.



Prática Guiada — Passo 1: Instalação

Baixe e instale o Node.js LTS em nodejs.org. Baixe e instale o VS Code em code.visualstudio.com. Após instalar, verifique as versões no terminal:

```
node -v
```

```
npm -v
```

Se aparecerem números de versão (ex: v20.11.0 e 10.2.4), está tudo certo.

Passo 2 — Criando a pasta do projeto

Crie a pasta principal do nosso projeto, que vamos evoluir ao longo de TODAS as aulas do curso:

```
mkdir biblioteca-node  
cd biblioteca-node  
code .
```

Passo 3 — Inicializando o projeto com NPM

O NPM (Node Package Manager) é o gerenciador de pacotes do Node.js. O comando abaixo cria o arquivo package.json, que é a 'certidão de nascimento' do projeto — nele ficam registrados nome, versão, scripts e dependências.

```
npm init -y
```

O parâmetro -y aceita todas as respostas padrão automaticamente. Abra o package.json gerado e observe sua estrutura:

package.json

```
{
  "name": "biblioteca-node",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "license": "ISC"
}
```

Passo 4 — Criando o index.js

Crie o arquivo principal do projeto:

index.js

```
console.log("Servidor da Biblioteca iniciado!");

const nomeDoProjeto = "Sistema de Gerenciamento de Biblioteca";
const versao = "1.0.0";

console.log(`Projeto: ${nomeDoProjeto} - v${versao}`);
```

Execute com:

```
node index.js
```

Você deve ver as duas mensagens no terminal. Isso prova que o Node.js está executando JavaScript puro, fora de qualquer navegador.

Passo 5 — Criando módulos separados

Programas profissionais nunca ficam em um único arquivo. Vamos criar módulos (arquivos separados por responsabilidade) e aprender a importar um dentro do outro.

Crie o arquivo `utils.js` com funções utilitárias:

`utils.js`

```
function saudacao(nome) {  
  return `Olá, ${nome}! Bem-vindo ao sistema da biblioteca.`;  
}  
  
function formatarData(data = new Date()) {  
  return data.toLocaleDateString('pt-BR');  
}  
  
// Exportando as funções para que outros arquivos possam usá-las  
module.exports = { saudacao, formatarData };
```

Crie o arquivo `config.js` com as configurações gerais do projeto:

`config.js`

```
module.exports = {  
  nomeProjeto: "Sistema de Gerenciamento de Biblioteca",  
  versao: "1.0.0",  
  porta: 3000,  
  ambiente: "desenvolvimento",  
};
```

Passo 6 — Utilizando require()

A função `require()` é como usamos, no Node.js, o conteúdo exportado por outro arquivo (`module.exports`). Atualize o `index.js`:

index.js

```
const { saudacao, formatarData } = require('./utils');
const config = require('./config');

console.log(saudacao('Professor'));
console.log(`Data de hoje: ${formatarData()}`);
console.log(`Rodando o projeto: ${config.nomeProjeto}`);
console.log(`Porta configurada: ${config.porta}`);
```

Execute novamente com `node index.js` e observe como os três arquivos conversam entre si. Esse é o princípio da modularização, que vamos usar durante todo o curso (e é a base da arquitetura MVC que veremos na Aula 03).

Passo 7 — Organização do projeto

Ao final da aula, a estrutura de pastas deve estar assim:

```
biblioteca-node/
├── index.js
├── utils.js
├── config.js
└── package.json
```



Desafio da Aula 01

Desafio: Calculadora com Módulos

Crie um arquivo `calculadora.js` que exporte 4 funções: `somar(a, b)`, `subtrair(a, b)`, `multiplicar(a, b)` e `dividir(a, b)`. No `index.js`, importe o módulo com `require()` e teste as 4 operações imprimindo o resultado no console. Dica: na função `dividir`, trate o caso de divisão por zero retornando uma mensagem de erro amigável em vez de deixar o programa quebrar.



✓ Ao final desta aula você entende o que é Node.js, como ele executa JavaScript, o papel do Event Loop, e já sabe organizar um projeto em módulos — a base de tudo que construiremos daqui em diante.



Aula 02 — Express.js

Duração total: 3 horas.

Objetivos

Criar o primeiro servidor web profissional utilizando o framework Express.js, entendendo o ciclo completo de requisição e resposta HTTP.

Teoria — HTTP, Request e Response

HTTP (HyperText Transfer Protocol) é o protocolo que define como cliente e servidor trocam mensagens na web. Toda comunicação segue o ciclo:

Request (Requisição)

O cliente envia uma Request contendo: método (GET, POST...), URL, cabeçalhos (headers) e, às vezes, um corpo (body).

Response (Resposta)

O servidor processa e devolve uma Response contendo: status code (200, 404, 500...), headers e um corpo de resposta (HTML, JSON, etc.).

Rotas

Uma rota é a combinação entre um método HTTP e um caminho (path) que o servidor sabe responder. Por exemplo: GET /livros lista os livros, POST /livros cadastra um livro novo. Cada rota tem uma função responsável por tratá-la.

Express.js

Node.js puro já permite criar servidores HTTP (módulo http), mas de forma verbosa e manual. O Express é um framework minimalista que simplifica drasticamente a criação de rotas, o tratamento de requisições e respostas, e a organização de middlewares. É o framework mais usado do ecossistema Node.js e será a espinha dorsal do nosso projeto.

Middleware

Middleware é uma função que fica 'no meio do caminho' entre a requisição chegar e a resposta ser enviada. Ele pode ler dados, modificar a requisição, validar informações, bloquear o acesso, ou simplesmente registrar um log — e então decide se passa a requisição adiante (next()) ou interrompe o ciclo. Vamos usar middlewares extensivamente: para interpretar JSON, autenticar usuários e tratar erros.

Respostas em JSON

JSON (JavaScript Object Notation) é o formato padrão para troca de dados entre APIs modernas. É leve, legível e nativo do JavaScript. Toda nossa API (a partir da Aula 08) devolverá dados em JSON.

Prática Guiada — Express

Passo 1 — Instalando o Express

Dentro da pasta biblioteca-node, instale o Express como dependência do projeto:

```
npm install express
```

Isso cria a pasta node_modules e adiciona 'express' na lista de dependencies do package.json.

Passo 2 — Criando o server.js

server.js

```
const express = require('express');
const app = express();
const PORTA = 3000;

app.listen(PORTA, () => {
  console.log(`Servidor rodando em http://localhost:${PORTA}`);
});
```

Execute:

```
node server.js
```

Acesse <http://localhost:3000> no navegador. Você ainda verá um erro 'Cannot GET /' — isso é esperado, pois ainda não criamos nenhuma rota.

Passo 3 — Criando as primeiras rotas

server.js

```
const express = require('express');
const app = express();
const PORTA = 3000;

app.get('/', (req, res) => {
  res.send('Bem-vindo ao Sistema de Gerenciamento de Biblioteca!');
});

app.get('/sobre', (req, res) => {
  res.send('Este é um projeto acadêmico para gerenciar livros, usuários e empréstimos.');
```

```
});

app.get('/contato', (req, res) => {
  res.send('Fale com a biblioteca pelo e-mail: contato@biblioteca.com');
```

```
});

app.listen(PORTA, () => {
  console.log(`Servidor rodando em http://localhost:${PORTA}`);
});
```

Reinicie o servidor (Ctrl+C e node server.js novamente) e teste as 3 rotas no navegador: /, /sobre e /contato.

  Dica: nodemon — Reiniciar o servidor manualmente a cada alteração é cansativo. Instale o nodemon como dependência de desenvolvimento (npm install -D nodemon) e crie um script no package.json: "dev": "nodemon server.js". Rode com npm run dev — o servidor reinicia sozinho a cada alteração salva.

Passo 4 — Retornando JSON

Além de texto puro (`res.send()`), o Express permite responder diretamente com JSON usando `res.json()`. Isso é essencial para quando começarmos a construir a API de verdade.

server.js (trecho)

```
app.get('/sobre', (req, res) => {  
  res.json({  
    projeto: 'Sistema de Gerenciamento de Biblioteca',  
    versao: '1.0.0',  
    autor: 'Turma de Node.js',  
  });  
});
```

Acesse <http://localhost:3000/sobre> novamente e observe que o navegador agora recebe um objeto JSON, não mais um texto simples.

Passo 5 — Middleware de log simples

Vamos criar nosso primeiro middleware, que registra no console todas as requisições recebidas pelo servidor:

server.js (adicionar antes das rotas)

```
app.use((req, res, next) => {  
  const agora = new Date().toLocaleTimeString('pt-BR');  
  console.log(`[${agora}] ${req.method} ${req.url}`);  
  next(); // repassa a requisição para a próxima rota/middleware  
});
```

Esse middleware deve ser registrado com `app.use()` ANTES das rotas, pois a ordem de execução no Express segue exatamente a ordem em que o código foi escrito.



Desafio da Aula 02

Desafio: API de Produtos

Crie um novo arquivo `produtos.js` (ou continue no mesmo `server.js`) com um array de produtos em memória (`id`, `nome`, `preço`). Crie as rotas: `GET /produtos` (lista todos), `GET /produtos/:id` (retorna um produto pelo `id`, usando `req.params`). Todas as respostas devem ser em JSON com `res.json()`. Se o produto não existir, responda com status 404 usando `res.status(404).json({ erro: 'Produto não encontrado' })`.



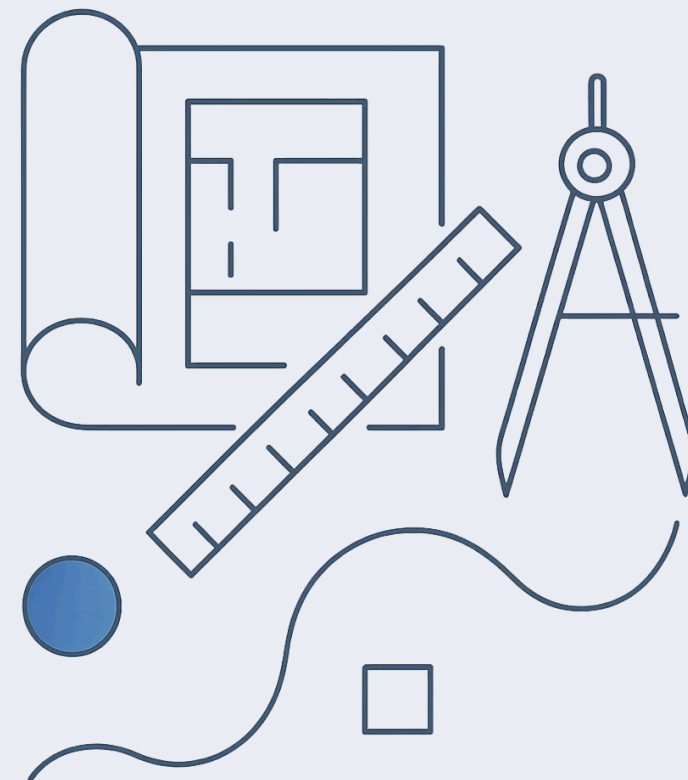
✓ Ao final desta aula você tem seu primeiro servidor web funcionando com Express, rotas GET, respostas em JSON e seu primeiro middleware.

Aula 03 — MVC (Model-View-Controller)

Duração total: 3 horas.

Objetivos

Organizar a aplicação seguindo o padrão de arquitetura MVC, usado pela imensa maioria dos sistemas profissionais no mercado.



Teoria — O que é MVC?

MVC significa Model-View-Controller e é um padrão de arquitetura que separa a aplicação em três responsabilidades bem definidas:

Model

representa os dados e as regras de negócio (ex: um livro, um usuário, um empréstimo).

View

representa o que o usuário vê (telas, HTML, EJS — veremos a partir da Aula 07).

Controller

recebe as requisições, conversa com o Model, decide o que fazer e devolve a resposta (chamando a View ou retornando JSON).

O objetivo do MVC é separar responsabilidades para que o código fique organizado, fácil de manter e fácil de escalar em equipe. Sem isso, é comum ver projetos com centenas de linhas dentro de um único arquivo `server.js` — o chamado 'espaguete de código'.

Rotas no MVC

As rotas (routes) ficam isoladas em arquivos próprios, cuja única responsabilidade é dizer: 'quando chegar tal requisição, chame tal função do Controller'. As rotas não têm lógica de negócio nenhuma.

Controllers

Os controllers concentram a lógica de cada ação: buscar dados, validar informações, e retornar a resposta adequada. É aqui que a 'inteligência' de cada operação do CRUD (Create, Read, Update, Delete) mora.

Organização de pastas

A estrutura de pastas que vamos adotar para o projeto biblioteca-node, e manter até o final do curso, é:

```
biblioteca-node/
├── controllers/
│   └── livroController.js
├── routes/
│   └── livroRoutes.js
├── models/
│   └── Livro.js
├── views/
├── public/
├── server.js
└── package.json
```



Prática Guiada — MVC

Passo 1 — Criando as pastas

```
mkdir controllers routes models views public
```

Passo 2 — Criando o Model de Livro (em memória)

Por enquanto, sem banco de dados (isso vem na Aula 04), vamos simular nossos dados com um array em memória dentro do Model.

models/Livro.js

```
// Nosso "banco de dados" temporário, em memória
let livros = [
  { id: 1, titulo: 'Dom Casmurro', autor: 'Machado de Assis', categoria: 'Romance' },
  { id: 2, titulo: 'O Cortiço', autor: 'Aluísio Azevedo', categoria: 'Romance' },
];
let proximoid = 3;

function listarTodos() { return livros; }
function buscarPorId(id) { return livros.find((livro) => livro.id === Number(id)); }
function criar(dados) {
  const novoLivro = { id: proximoid++, ...dados };
  livros.push(novoLivro);
  return novoLivro;
}
function atualizar(id, dados) {
  const livro = buscarPorId(id);
  if (!livro) return null;
  Object.assign(livro, dados);
  return livro;
}
function remover(id) {
  const existe = buscarPorId(id);
  if (!existe) return false;
  livros = livros.filter((livro) => livro.id !== Number(id));
  return true;
}

module.exports = { listarTodos, buscarPorId, criar, atualizar, remover };
```

Passo 3 — Criando o Controller

controllers/livroController.js

```
const Livro = require('../models/Livro');

function index(req, res) { res.json(Livro.listarTodos()); }

function show(req, res) {
  const livro = Livro.buscarPorId(req.params.id);
  if (!livro) { return res.status(404).json({ erro: 'Livro não encontrado' }); }
  res.json(livro);
}

function store(req, res) {
  const { titulo, autor, categoria } = req.body;
  if (!titulo || !autor) { return res.status(400).json({ erro: 'Título e autor são obrigatórios' }); }
  const novoLivro = Livro.criar({ titulo, autor, categoria });
  res.status(201).json(novoLivro);
}

function update(req, res) {
  const livroAtualizado = Livro.atualizar(req.params.id, req.body);
  if (!livroAtualizado) { return res.status(404).json({ erro: 'Livro não encontrado' }); }
  res.json(livroAtualizado);
}

function destroy(req, res) {
  const removido = Livro.remover(req.params.id);
  if (!removido) { return res.status(404).json({ erro: 'Livro não encontrado' }); }
  res.status(204).send();
}

module.exports = { index, show, store, update, destroy };
```

Passo 4 e 5 — Rotas e server.js

Passo 4 — Criando as Rotas

routes/livroRoutes.js

```
const express = require('express');
const router = express.Router();
const livroController = require('../controllers/livroController');

router.get('/', livroController.index);
router.get('/:id', livroController.show);
router.post('/', livroController.store);
router.put('/:id', livroController.update);
router.delete('/:id', livroController.destroy);

module.exports = router;
```

Passo 5 — Conectando tudo no server.js

server.js

```
const express = require('express');
const app = express();
const PORTA = 3000;

app.use(express.json()); // middleware que interpreta JSON no corpo das requisições

const livroRoutes = require('./routes/livroRoutes');
app.use('/livros', livroRoutes);

app.listen(PORTA, () => {
  console.log(`Servidor rodando em http://localhost:${PORTA}`);
});
```

Note o `app.use(express.json())`: sem esse middleware, o Express não consegue ler o `req.body` quando o cliente envia dados em JSON (essencial para os métodos POST e PUT). Ele deve vir antes das rotas.

Teste com o Postman/Insomnia/Thunder Client (ou vamos usar formalmente na Aula 08): GET `/livros`, GET `/livros/1`, POST `/livros` com body JSON, PUT `/livros/1`, DELETE `/livros/1`.

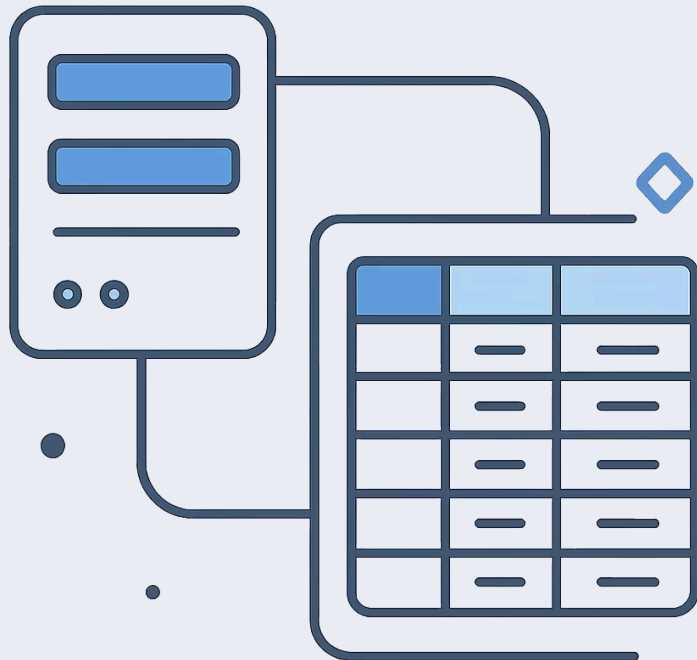


Desafio da Aula 03

Desafio: Adicionar Categoria ao Livro

O campo 'categoria' já existe no Model, mas vamos garantir a validação: no Controller (store), exija que categoria também seja obrigatória. Depois, crie uma nova rota GET /livros/categoria/:categoria no livroRoutes.js e uma função filtrarPorCategoria no Model + Controller, que retorne apenas os livros daquela categoria (ex: GET /livros/categoria/Romance).

- 👍 ☒ Ao final desta aula seu projeto está organizado em MVC, com um CRUD completo de livros funcionando em memória — pronto para ganhar um banco de dados de verdade na próxima aula.



Aula 04 — Banco de Dados MySQL

Duração total: 3 horas.

Objetivos

Substituir o array em memória por persistência real de dados, utilizando um banco de dados relacional (MySQL).

Teoria — Banco de Dados Relacional

Um banco de dados relacional organiza informações em tabelas, compostas por linhas (registros) e colunas (campos/atributos). As tabelas podem se relacionar entre si através de chaves — é isso que torna o modelo 'relacional'. No nosso sistema, teremos tabelas como livros, usuarios e empréstimos, todas relacionadas entre si.

MER — Modelo Entidade-Relacionamento

Antes de criar as tabelas, um bom profissional desenha o MER: um diagrama que representa as entidades (livro, usuário, empréstimo) e como elas se relacionam. Para o nosso projeto:

- Um USUÁRIO pode fazer vários EMPRÉSTIMOS (1:N).
- Um LIVRO pode estar em vários EMPRÉSTIMOS ao longo do tempo (1:N).
- Um EMPRÉSTIMO pertence a exatamente um USUÁRIO e a exatamente um LIVRO (N:1 para cada lado).

SQL — a linguagem dos bancos relacionais

SQL (Structured Query Language) é a linguagem universal para conversar com bancos relacionais como MySQL, PostgreSQL e SQL Server. Os quatro comandos fundamentais que usaremos hoje formam o acrônimo CRUD:

CREATE / INSERT

criar estruturas e inserir dados.

SELECT

consultar (ler) dados.

UPDATE

atualizar dados existentes.

DELETE

remover dados.

Prática Guiada — MySQL

Passo 1 — Instalando o MySQL

Baixe e instale o MySQL Community Server (dev.mysql.com) e, opcionalmente, o MySQL Workbench para gerenciar visualmente o banco. Alternativa mais simples: instale o XAMPP, que já vem com MySQL configurado.

Passo 2 — Criando o banco de dados

Abra o MySQL Workbench (ou terminal mysql) e execute:

```
CREATE DATABASE biblioteca;  
USE biblioteca;
```

Passo 3 — Criando a tabela de livros

```
CREATE TABLE livros (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  titulo VARCHAR(150) NOT NULL,  
  autor VARCHAR(120) NOT NULL,  
  categoria VARCHAR(60),  
  disponivel BOOLEAN DEFAULT TRUE,  
  criado_em DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Passo 4 — SQL: INSERT, SELECT, UPDATE, DELETE

```
-- INSERT: adicionar registros
INSERT INTO livros (titulo, autor, categoria) VALUES
('Dom Casmurro', 'Machado de Assis', 'Romance'),
('O Cortiço', 'Aluísio Azevedo', 'Romance'),
('Capitães da Areia', 'Jorge Amado', 'Romance');

-- SELECT: consultar todos
SELECT * FROM livros;

-- SELECT com filtro
SELECT * FROM livros WHERE categoria = 'Romance';

-- UPDATE: atualizar um registro
UPDATE livros SET disponivel = FALSE WHERE id = 1;

-- DELETE: remover um registro
DELETE FROM livros WHERE id = 3;
```

Passo 5 — Conectando o Node.js ao MySQL

Instale o driver mysql2, que permite ao Node.js se comunicar com o banco:

```
npm install mysql2
```

Crie um arquivo de configuração de conexão:

config/database.js

```
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'SUA_SENHA_AQUI',
  database: 'biblioteca',
});

module.exports = pool;
```

⚠️ ⚠️ Segurança: Nunca deixe senhas de banco de dados escritas diretamente no código em um projeto real. Use variáveis de ambiente com o pacote dotenv (npm install dotenv) e um arquivo .env (que nunca deve ser enviado ao GitHub). Por ora, para fins didáticos, vamos manter simples — mas guarde essa boa prática para o Projeto Final.

Passo 6 — Atualizando o Model para usar o banco

models/Livro.js (nova versão com MySQL)

```
const pool = require('../config/database');

async function listarTodos() {
  const [linhas] = await pool.query('SELECT * FROM livros');
  return linhas;
}

async function buscarPorId(id) {
  const [linhas] = await pool.query('SELECT * FROM livros WHERE id = ?', [id]);
  return linhas[0];
}

module.exports = { listarTodos, buscarPorId };
```

Repare no uso de '?' na query — isso é chamado de prepared statement (consulta parametrizada). NUNCA concatene strings diretamente numa query SQL (ex: `SELECT * FROM livros WHERE id = ${id}`), pois isso abre brecha para SQL Injection, um dos ataques mais comuns em sistemas web.

Note que agora as funções são async (assíncronas) — vamos aprofundar exatamente esse conceito na Aula 06. Por enquanto, ajuste o Controller para usar await antes de chamar cada função do Model, e transforme cada função do Controller em async.

Desafio da Aula 04

Desafio: Cadastrar 20 livros via SQL

Escreva um único comando `INSERT INTO livros (titulo, autor, categoria) VALUES (...)` que cadastre 20 livros diferentes de uma só vez (pode usar livros reais ou fictícios). Depois, escreva 3 consultas `SELECT` diferentes: uma que traga apenas livros de uma categoria específica, uma que ordene os livros por título (`ORDER BY titulo`), e uma que conte quantos livros existem por categoria (dica: pesquise sobre `GROUP BY` e `COUNT`).

✔️ ☒ Ao final desta aula seu projeto persiste dados de verdade em um banco MySQL, e você domina os 4 pilares do SQL: CREATE, INSERT, UPDATE, DELETE e SELECT.



Aula 05 — ORM (Sequelize)

Duração total: 3 horas.

Objetivos

Substituir as queries SQL escritas manualmente por um ORM profissional, ganhando produtividade, segurança e organização.

Teoria — O que é ORM?

ORM significa Object-Relational Mapping (Mapeamento Objeto-Relacional). É uma biblioteca que permite trabalhar com o banco de dados usando objetos e funções JavaScript, em vez de escrever SQL manualmente. Um Model no ORM representa uma tabela, e cada instância desse Model representa uma linha da tabela.

Vantagens: menos código repetitivo, proteção automática contra SQL Injection, e um código mais legível e portátil entre diferentes bancos de dados (MySQL, PostgreSQL, SQLite...).

Models no Sequelize

No Sequelize (o ORM mais popular do ecossistema Node.js), cada Model é uma classe JavaScript que define as colunas da tabela e seus tipos. O Sequelize se encarrega de gerar o SQL por trás dos panos.

Migration

Migrations são como um 'histórico de versões' da estrutura do banco de dados. Em vez de alterar tabelas manualmente, você escreve arquivos de migration que descrevem cada mudança (criar tabela, adicionar coluna, etc.), permitindo que qualquer pessoa da equipe recrie o banco do zero rodando as migrations em ordem.

Sincronização (sync)

Para fins didáticos e agilidade em sala, vamos usar o método `sequelize.sync()`, que cria automaticamente as tabelas com base nos Models definidos. Em produção, o ideal é usar Migrations (`sequelize-cli`), que dão mais controle e segurança sobre alterações no banco.



Prática Guiada — Sequelize

Passo 1 — Instalando o Sequelize

```
npm install sequelize mysql2
```

Passo 2 — Configurando a conexão

config/sequelize.js

```
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize('biblioteca', 'root', 'SUA_SENHA_AQUI', {
  host: 'localhost',
  dialect: 'mysql',
  logging: false, // desliga o log de todas as queries geradas no console
});

module.exports = sequelize;
```

Passo 3 — Criando o Model Livro com Sequelize

models/Livro.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/sequelize');

const Livro = sequelize.define('Livro', {
  titulo: { type: DataTypes.STRING, allowNull: false },
  autor: { type: DataTypes.STRING, allowNull: false },
  categoria: { type: DataTypes.STRING },
  disponivel: { type: DataTypes.BOOLEAN, defaultValue: true },
}, {
  tableName: 'livros',
  timestamps: true, // cria createdAt e updatedAt automaticamente
});

module.exports = Livro;
```

Passo 4 e 5 — Sync e Controller com Sequelize

Passo 4 — Sincronizando com o banco

server.js

```
const express = require('express');
const app = express();
const sequelize = require('./config/sequelize');

app.use(express.json());

sequelize.sync()
  .then(() => console.log('Tabelas sincronizadas com sucesso!'))
  .catch((erro) => console.error('Erro ao sincronizar:', erro));

const livroRoutes = require('./routes/livroRoutes');
app.use('/livros', livroRoutes);

app.listen(3000, () => console.log('Servidor rodando na porta 3000'));
```

Passo 5 — Atualizando o Controller para usar os métodos do Sequelize

controllers/livroController.js

```
const Livro = require('../models/Livro');

async function index(req, res) {
  const livros = await Livro.findAll();
  res.json(livros);
}

async function show(req, res) {
  const livro = await Livro.findByPk(req.params.id);
  if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
  res.json(livro);
}

async function store(req, res) {
  const novoLivro = await Livro.create(req.body);
  res.status(201).json(novoLivro);
}

async function update(req, res) {
  const livro = await Livro.findByPk(req.params.id);
  if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
  await livro.update(req.body);
  res.json(livro);
}

async function destroy(req, res) {
  const livro = await Livro.findByPk(req.params.id);
  if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
  await livro.destroy();
  res.status(204).send();
}

module.exports = { index, show, store, update, destroy };
```

Compare com a versão da Aula 04: o código ficou mais curto, mais legível, e não escrevemos SQL manualmente em nenhum momento. `findAll()`, `findByPk()`, `create()`, `update()` e `destroy()` são os métodos essenciais do Sequelize que usaremos o curso inteiro.



Desafio da Aula 05

Desafio: Criar o Model Usuário

Crie `models/Usuario.js` com os campos: `nome` (string, obrigatório), `email` (string, obrigatório e único — use `unique: true`), `senha` (string, obrigatório) e `tipo` (string, com `defaultValue 'aluno'`). Crie o controller e as rotas seguindo o mesmo padrão do Livro (`index`, `show`, `store`, `update`, `destroy`), disponíveis em `/usuarios`. Teste o CRUD completo com o Postman/Insomnia.

- ✔ ☒ Ao final desta aula você domina o uso de um ORM profissional, com Models bem definidos e um CRUD completo rodando sobre o Sequelize — muito mais próximo do que se vê no mercado de trabalho.

Aula 06 — Async / Await

Duração total: 3 horas.

Objetivos

Dominar de fato a programação assíncrona em JavaScript — a peça-chave que já vínhamos usando desde a Aula 04, mas que agora vamos entender em profundidade.





Teoria — Callback, Promise e Async/Await

Callback

A forma mais antiga de lidar com código assíncrono em JavaScript é o callback: uma função passada como argumento, que é executada somente quando a operação assíncrona termina.

```
buscarLivro(1, function (erro, livro) {  
  if (erro) {  
    console.error('Deu erro:', erro);  
  } else {  
    console.log('Livro encontrado:', livro);  
  }  
});
```

O problema do callback é que, quando temos várias operações assíncronas encadeadas, o código cresce em profundidade e vira o famoso 'Callback Hell' (inferno dos callbacks) — difícil de ler e de manter.

Promise

A Promise foi criada para resolver esse problema. Ela representa um valor que estará disponível no futuro — podendo ser resolvido (sucesso) ou rejeitado (erro).

```
buscarLivro(1)  
  .then((livro) => console.log('Livro encontrado:', livro))  
  .catch((erro) => console.error('Deu erro:', erro));
```

Isso já é mais legível, mas quando encadeamos muitos `.then()`, ainda fica repetitivo.

Async / Await

Async/Await é uma forma de escrever código assíncrono como se fosse síncrono — muito mais legível. Uma função marcada com `async` sempre retorna uma Promise. Dentro dela, usamos `await` para 'pausar' a execução daquela função até a Promise ser resolvida, sem bloquear o restante da aplicação (o Event Loop continua livre para atender outras requisições).

```
async function exibirLivro() {  
  const livro = await buscarLivro(1);  
  console.log('Livro encontrado:', livro);  
}
```

Try / Catch

Como o `await` 'esconde' o `.then()/.catch()` da Promise, usamos o bloco `try/catch` tradicional do JavaScript para capturar erros em código assíncrono com `async/await`.

```
async function exibirLivro() {  
  try {  
    const livro = await buscarLivro(1);  
    console.log('Livro encontrado:', livro);  
  } catch (erro) {  
    console.error('Deu erro:', erro.message);  
  }  
}
```

Prática Guiada — Async/Await

Passo 1 — Comparando o antes e depois

Vamos revisitar o Controller da Aula 03 (versão em memória, sem async) e comparar com a versão do Sequelize (Aula 05, já com async/await), para fixar a diferença na prática.

Antes (sem Promises)

```
// ANTES — Aula 03, síncrono, dados em memória
function index(req, res) {
  res.json(Livro.listarTodos());
}
```

Com .then()

```
// USANDO .then() — funcionaria, mas é menos legível
function index(req, res) {
  Livro.findAll()
    .then((livros) => res.json(livros))
    .catch((erro) => res.status(500).json({ erro: erro.message }));
}
```

Depois (com async/await)

```
// DEPOIS — async/await, muito mais legível
async function index(req, res) {
  try {
    const livros = await Livro.findAll();
    res.json(livros);
  } catch (erro) {
    res.status(500).json({ erro: erro.message });
  }
}
```

Passo 2 — CRUD com try/catch completo

Vamos garantir que absolutamente todas as funções do controllers/livroController.js tenham tratamento de erro adequado (algo que pulamos rapidamente na Aula 05 para focar no Sequelize):

controllers/livroController.js (revisado)

```
const Livro = require('../models/Livro');

async function index(req, res) {
  try {
    const livros = await Livro.findAll();
    res.json(livros);
  } catch (erro) {
    res.status(500).json({ erro: 'Erro ao buscar livros', detalhes: erro.message });
  }
}

async function show(req, res) {
  try {
    const livro = await Livro.findByPk(req.params.id);
    if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
    res.json(livro);
  } catch (erro) {
    res.status(500).json({ erro: 'Erro ao buscar livro', detalhes: erro.message });
  }
}

async function store(req, res) {
  try {
    const novoLivro = await Livro.create(req.body);
    res.status(201).json(novoLivro);
  } catch (erro) {
    res.status(400).json({ erro: 'Erro ao criar livro', detalhes: erro.message });
  }
}

module.exports = { index, show, store /* ...update, destroy seguem o mesmo padrão */ };
```

  Por que status 400 na criação e 500 na busca? Status 400 (Bad Request) indica que o erro foi causado por dados inválidos enviados pelo cliente (ex: campo obrigatório faltando). Status 500 (Internal Server Error) indica uma falha do próprio servidor (ex: banco de dados fora do ar). Diferenciar esses status ajuda quem consome a API a entender de quem foi a 'culpa' do erro.

Passo 3 — Promise.all() para operações em paralelo

Às vezes precisamos buscar várias coisas ao mesmo tempo, sem que uma espere a outra. Para isso usamos Promise.all():

controllers/dashboardController.js

```
async function dashboard(req, res) {
  try {
    const [totalLivros, totalUsuarios] = await Promise.all([
      Livro.count(),
      Usuario.count(),
    ]);
    res.json({ totalLivros, totalUsuarios });
  } catch (erro) {
    res.status(500).json({ erro: erro.message });
  }
}
```

Isso é muito mais eficiente do que fazer 'await Livro.count()' e depois 'await Usuario.count()' separadamente, pois as duas consultas rodam ao mesmo tempo. Vamos usar exatamente esse recurso na Aula 10, no Dashboard do Projeto Final.



Desafio da Aula 06

Desafio: Buscar livro por ID com tratamento completo

Crie uma função assíncrona buscarLivroComDetalhes(id) no Model Livro que: 1) busca o livro pelo ID; 2) se não encontrar, lança um erro customizado (throw new Error('Livro não encontrado')); 3) se encontrar, retorna o livro. No Controller, chame essa função dentro de um try/catch e trate a mensagem de erro lançada, devolvendo o status HTTP correto (404 se a mensagem for 'Livro não encontrado', 500 para qualquer outro erro).

- ✔ ☒ Ao final desta aula você entende profundamente a diferença entre callback, Promise e async/await, e sabe tratar erros assíncronos de forma profissional em toda a aplicação.

Aula 07 — EJS (Views Dinâmicas)

Duração total: 3 horas.

Objetivos

Criar páginas HTML dinâmicas, que mudam de acordo com os dados vindos do banco — o famoso 'painel web' do nosso sistema.

Teoria

Template Engine

Um template engine (motor de templates) permite misturar HTML com lógica de programação (variáveis, loops, condicionais), gerando páginas dinâmicas no servidor antes de enviá-las ao navegador. Isso é chamado de Server-Side Rendering (SSR).

EJS (Embedded JavaScript)

EJS é um template engine simples que usa a própria sintaxe do JavaScript dentro do HTML, através das tags `<% %>` (lógica, sem imprimir nada), `<%= %>` (imprime um valor escapando HTML, mais seguro) e `<%- %>` (imprime sem escapar, útil para incluir outros arquivos EJS).

Views

No padrão MVC, as Views ficam na pasta `views/` e são responsáveis apenas por exibir dados — nunca devem conter lógica de negócio (isso é papel do Controller e do Model).

Layout

Para não repetir o mesmo cabeçalho/rodapé em toda página, usamos `partials` (arquivos EJS reutilizáveis, como `header.ejs` e `footer.ejs`) incluídos em cada view com `<%- include('partials/header') %>`.



Prática Guiada — EJS

Passo 1 — Instalando o EJS

```
npm install ejs
```

Passo 2 — Configurando o Express para usar EJS

server.js (adicionar no topo)

```
const express = require('express');
const app = express();
const path = require('path');

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());
app.use(express.urlencoded({ extended: true })); // interpreta dados de formulários HTML
```

Passo 3 — Criando os Partials (Layout)

views/partials/header.ejs

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Biblioteca - Painel</title>
  <link rel="stylesheet" href="/css/style.css">
</head>
<body>
<header>
<h1>
```

Passo 4, 5 e 6 — Views de Livros

Passo 4 — Tela Home (Listagem)

views/livros/index.ejs

```
<%- include('partials/header') %>
<h2>Livros cadastrados</h2>
<table>
  <thead>
    <tr><th>ID</th><th>Título</th><th>Autor</th><th>Categoria</th><th>Ações</th></tr>
  </thead>
  <tbody>
    <% livros.forEach(livro => { %>
      <tr>
        <td><%= livro.id %></td>
        <td><%= livro.titulo %></td>
        <td><%= livro.autor %></td>
        <td><%= livro.categoria %></td>
        <td>
          <a href="/livros/<%= livro.id %>/editar">Editar</a>
          <form action="/livros/<%= livro.id %>/excluir" method="POST" style="display:inline">
            <button type="submit">Excluir</button>
          </form>
        </td>
      </tr>
    <% }) %>
  </tbody>
</table>
<%- include('partials/footer') %>
```

Passo 5 — Tela de Cadastro (Formulário)

views/livros/novo.ejs

```
<%- include('partials/header') %>
<h2>Cadastrar novo livro</h2>
<form action="/livros" method="POST">
  <label>Título:</label>
  <input type="text" name="titulo" required>
  <label>Autor:</label>
  <input type="text" name="autor" required>
  <label>Categoria:</label>
  <input type="text" name="categoria">
  <button type="submit">Salvar</button>
</form>
<%- include('partials/footer') %>
```

Passo 6 — Tela de Edição

views/livros/editar.ejs

```
<%- include('partials/header') %>
<h2>Editar livro</h2>
<form action="/livros/<%= livro.id %>/editar" method="POST">
  <label>Título:</label>
  <input type="text" name="titulo" value="<%= livro.titulo %>" required>
  <label>Autor:</label>
  <input type="text" name="autor" value="<%= livro.autor %>" required>
  <label>Categoria:</label>
  <input type="text" name="categoria" value="<%= livro.categoria %>">
  <button type="submit">Atualizar</button>
</form>
<%- include('partials/footer') %>
```

Passo 7 — Controller e Rotas para o Painel

Note que, para o painel web (EJS), usamos `res.render()` ao invés de `res.json()`. Formulários HTML tradicionais não sabem enviar PUT/DELETE, então usamos POST para tudo e simulamos as outras ações por rota:

controllers/livroController.js (versão para o painel)

```
async function index(req, res) {
  const livros = await Livro.findAll();
  res.render('livros/index', { livros });
}
function formularioNovo(req, res) { res.render('livros/novo'); }
async function store(req, res) {
  await Livro.create(req.body);
  res.redirect('/livros');
}
async function formularioEditar(req, res) {
  const livro = await Livro.findByPk(req.params.id);
  res.render('livros/editar', { livro });
}
async function update(req, res) {
  const livro = await Livro.findByPk(req.params.id);
  await livro.update(req.body);
  res.redirect('/livros');
}
async function destroy(req, res) {
  const livro = await Livro.findByPk(req.params.id);
  await livro.destroy();
  res.redirect('/livros');
}
module.exports = { index, formularioNovo, store, formularioEditar, update, destroy };
```

routes/livroRoutes.js

```
router.get('/', livroController.index);
router.get('/novo', livroController.formularioNovo);
router.post('/', livroController.store);
router.get('/:id/editar', livroController.formularioEditar);
router.post('/:id/editar', livroController.update);
router.post('/:id/excluir', livroController.destroy);
```

  Painel Web x API REST: Perceba que, a partir de agora, temos DUAS formas de expor os mesmos dados: via Views EJS (para humanos navegarem pelo painel) e via JSON (para outros sistemas consumirem, como veremos na Aula 08). É comum separar isso em rotas diferentes, ex: /livros (painel HTML) e /api/livros (API REST em JSON) — ambos usando o mesmo Model por trás.



Desafio da Aula 07

Desafio: Página de Detalhes

Crie a rota GET /livros/:id que renderiza uma nova view `views/livros/detalhes.ejs`, mostrando todas as informações do livro (título, autor, categoria, disponibilidade e data de cadastro) de forma mais completa que a tabela da listagem. Adicione um link 'Ver detalhes' em cada linha da tabela de listagem apontando para essa nova página.

  Ao final desta aula seu sistema tem um painel web completo e navegável — listagem, cadastro, edição e exclusão de livros, tudo através de páginas HTML renderizadas dinamicamente pelo servidor.

Aula 08 — REST API Completa

Duração total: 3 horas.

Objetivos

Formalizar e completar nossa API seguindo os princípios REST, disponibilizando os dados da biblioteca para qualquer cliente (painel web, app mobile, outro sistema).

Teoria — O que é REST?

REST (Representational State Transfer) é um estilo arquitetural para construir APIs, baseado em alguns princípios centrais: cada recurso (livro, usuário, empréstimo) tem uma URL própria; usamos os verbos HTTP para indicar a ação desejada sobre aquele recurso; e a comunicação é stateless (sem estado) — cada requisição contém todas as informações necessárias, o servidor não guarda 'memória' de requisições anteriores.

Verbos HTTP e Status Codes

Verbos HTTP e o CRUD

Cada verbo HTTP representa uma ação sobre o recurso:

GET /livros lista todos os livros (Read)	GET /livros/:id busca um livro específico (Read)
POST /livros cria um novo livro (Create)	PUT /livros/:id atualiza um livro por completo (Update)
PATCH /livros/:id atualiza parcialmente um livro (Update parcial)	DELETE /livros/:id remove um livro (Delete)

Status HTTP

Os códigos de status informam o resultado da requisição de forma padronizada:

200 OK sucesso genérico	201 Created recurso criado com sucesso
204 No Content sucesso, mas sem corpo de resposta (comum em DELETE)	400 Bad Request erro do cliente, dados inválidos
401 Unauthorized não autenticado	403 Forbidden autenticado, mas sem permissão
404 Not Found recurso não encontrado	500 Internal Server Error erro do servidor



Prática Guiada — REST API

Passo 1 — Separando API de Painel Web

Vamos organizar nossa API sob o prefixo /api, mantendo o painel EJS nas rotas 'normais' (como vimos na Aula 07):

server.js (trecho)

```
const livroRoutes = require('./routes/livroRoutes'); // painel EJS
const livroApiRoutes = require('./routes/api/livroApi'); // API REST em JSON

app.use('/livros', livroRoutes);
app.use('/api/livros', livroApiRoutes);
```

Passo 2 — Criando a API REST completa de Livros

routes/api/livroApi.js

```
const express = require('express');
const router = express.Router();
const Livro = require('../models/Livro');

// GET /api/livros
router.get('/', async (req, res) => {
  try {
    const livros = await Livro.findAll();
    res.status(200).json(livros);
  } catch (erro) { res.status(500).json({ erro: erro.message }); }
});

// GET /api/livros/:id
router.get('/:id', async (req, res) => {
  try {
    const livro = await Livro.findByPk(req.params.id);
    if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
    res.status(200).json(livro);
  } catch (erro) { res.status(500).json({ erro: erro.message }); }
});

// POST /api/livros
router.post('/', async (req, res) => {
  try {
    const { titulo, autor } = req.body;
    if (!titulo || !autor) { return res.status(400).json({ erro: 'Título e autor são obrigatórios' }); }
    const novoLivro = await Livro.create(req.body);
    res.status(201).json(novoLivro);
  } catch (erro) { res.status(400).json({ erro: erro.message }); }
});

// PUT /api/livros/:id
router.put('/:id', async (req, res) => {
  try {
    const livro = await Livro.findByPk(req.params.id);
    if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
    await livro.update(req.body);
    res.status(200).json(livro);
  } catch (erro) { res.status(400).json({ erro: erro.message }); }
});

// PATCH /api/livros/:id/disponibilidade
router.patch('/:id/disponibilidade', async (req, res) => {
  try {
    const livro = await Livro.findByPk(req.params.id);
    if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
    await livro.update({ disponivel: req.body.disponivel });
    res.status(200).json(livro);
  } catch (erro) { res.status(400).json({ erro: erro.message }); }
});

// DELETE /api/livros/:id
router.delete('/:id', async (req, res) => {
  try {
    const livro = await Livro.findByPk(req.params.id);
    if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
    await livro.destroy();
    res.status(204).send();
  } catch (erro) { res.status(500).json({ erro: erro.message }); }
});

module.exports = router;
```

Passo 3 — Testando com Postman / Insomnia / Thunder Client

Instale uma dessas ferramentas (Thunder Client é uma extensão dentro do próprio VS Code, ótima para quem está começando). Crie uma coleção de testes chamada 'Biblioteca API' com uma requisição para cada endpoint:

- GET `http://localhost:3000/api/livros`
- GET `http://localhost:3000/api/livros/1`
- POST `http://localhost:3000/api/livros` — Body (JSON): `{ "titulo": "1984", "autor": "George Orwell", "categoria": "Ficção" }`
- PUT `http://localhost:3000/api/livros/1` — Body (JSON) com os campos atualizados
- PATCH `http://localhost:3000/api/livros/1/disponibilidade` — Body: `{ "disponivel": false }`
- DELETE `http://localhost:3000/api/livros/1`

⚠️ ⚠️ **Lembrete: Content-Type** — Ao testar POST/PUT/PATCH no Postman/Insomnia/Thunder Client, sempre configure o header Content-Type: `application/json` e escolha 'raw/JSON' no corpo da requisição — senão o Express não conseguirá interpretar o `req.body` corretamente, mesmo com o middleware `express.json()` configurado.

Desafio da Aula 08

Desafio: API de Usuários

Replique a API REST completa (GET, GET por ID, POST, PUT, DELETE) para o recurso de Usuários, criando o arquivo `routes/api/usuarioApi.js` e registrando-o em `/api/usuarios`. Garanta que a senha do usuário nunca seja retornada nas respostas JSON (dica: use `{ attributes: { exclude: ['senha'] } }` nas consultas do Sequelize).

- ✅ ☒ Ao final desta aula você tem uma API REST completa e testável, seguindo os padrões de mercado — pronta para ser consumida por qualquer frontend, inclusive o que vamos construir na próxima aula.

Aula 09 — Integração Front-end + Back-end

Duração total: 3 horas.

Objetivos

Consumir a API REST criada na Aula 08 a partir de uma página HTML pura, usando JavaScript no navegador — simulando exatamente como um app mobile ou um frontend em React/Vue consumiria nosso backend.

Teoria — Axios x Fetch

Existem duas formas principais de fazer requisições HTTP a partir do navegador (ou de outro Node.js):

Fetch API

nativa do navegador, não precisa instalar nada, mas é um pouco mais verbosa (é preciso chamar `.json()` manualmente e tratar erros HTTP 'na mão').

Axios

biblioteca externa muito popular, com uma sintaxe mais enxuta, interceptors, e tratamento de erros mais amigável.

JSON no Frontend

Assim como no backend, o JSON é o formato de troca de dados entre o frontend e a API. O navegador envia JSON no corpo das requisições POST/PUT, e recebe JSON nas respostas, que são então usadas para montar dinamicamente o HTML da página (manipulação do DOM).



Prática Guiada — Integração

Passo 1 — Criando a página HTML de consumo

Dentro da pasta `public/`, crie uma página simples e independente do EJS, que vai consumir a API REST via JavaScript puro no navegador:

public/index.html

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Biblioteca - Consumo de API</title>
</head>
<body>
  <h1>Livros da Biblioteca</h1>
  <form id="form-livro">
    <input type="text" id="titulo" placeholder="Título" required>
    <input type="text" id="autor" placeholder="Autor" required>
    <input type="text" id="categoria" placeholder="Categoria">
    <button type="submit">Cadastrar</button>
  </form>
  <ul id="lista-livros"></ul>
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
  <script src="/js/app.js"></script>
</body>
</html>
```

Passo 2 — Consumindo GET /api/livros

public/js/app.js

```
const API_URL = 'http://localhost:3000/api/livros';

async function carregarLivros() {
  try {
    const resposta = await axios.get(API_URL);
    const livros = resposta.data;
    const lista = document.getElementById('lista-livros');
    lista.innerHTML = '';
    livros.forEach((livro) => {
      const item = document.createElement('li');
      item.innerHTML = `
        ${livro.titulo} - ${livro.autor}
        <button onclick="excluirLivro(${livro.id})">Excluir</button>
      `;
      lista.appendChild(item);
    });
  } catch (erro) {
    console.error('Erro ao carregar livros:', erro);
    alert('Não foi possível carregar os livros.');
```

Passo 3 e 4 — POST, PUT e DELETE com Axios

Passo 3 — Consumindo POST /api/livros

public/js/app.js (continuação)

```
document.getElementById('form-livro').addEventListener('submit', async (evento) => {
  evento.preventDefault(); // impede o recarregamento da página
  const novoLivro = {
    titulo: document.getElementById('titulo').value,
    autor: document.getElementById('autor').value,
    categoria: document.getElementById('categoria').value,
  };
  try {
    await axios.post(API_URL, novoLivro);
    document.getElementById('form-livro').reset();
    carregarLivros(); // recarrega a lista com o novo livro
  } catch (erro) {
    console.error('Erro ao cadastrar:', erro);
    alert('Erro ao cadastrar livro. Verifique os campos.');
```

Passo 4 — Consumindo PUT e DELETE

public/js/app.js (continuação)

```
async function excluirLivro(id) {
  const confirmar = confirm('Tem certeza que deseja excluir este livro?');
  if (!confirmar) return;
  try {
    await axios.delete(`${API_URL}/${id}`);
    carregarLivros();
  } catch (erro) {
    console.error('Erro ao excluir:', erro);
    alert('Erro ao excluir livro.');
```

```

  }
}

async function atualizarDisponibilidade(id, disponivel) {
  try {
    await axios.put(`${API_URL}/${id}`, { disponivel });
    carregarLivros();
  } catch (erro) {
    console.error('Erro ao atualizar:', erro);
  }
}
```



CORS: Se o frontend estivesse hospedado em um domínio diferente do backend, o navegador bloquearia essas requisições por segurança (política de CORS - Cross-Origin Resource Sharing). Como nosso HTML está sendo servido pelo próprio Express (express.static), não temos esse problema agora. Mas em projetos reais onde o frontend é separado (React, Vue rodando em outra porta), é preciso instalar e configurar o pacote cors no backend: npm install cors e app.use(cors()).

Desafio da Aula 09

Desafio: Cadastro de Livros com Axios

Adicione um botão 'Editar' em cada item da lista, que preenche o formulário com os dados do livro selecionado (busque com GET /api/livros/:id) e transforma o botão 'Cadastrar' em 'Atualizar' (chamando PUT ao invés de POST) enquanto um livro estiver em edição. Depois de atualizar, o formulário deve voltar ao modo normal de cadastro.



Ao final desta aula seu backend está sendo consumido de ponta a ponta por um frontend real, usando Axios — o mesmo padrão usado para integrar apps mobile, dashboards em React/Vue, ou qualquer outro cliente com sua API.

Aula 10 — Projeto Final: Sistema de Biblioteca Completo

Duração total: 4 horas (aula estendida para fechamento do curso).

Objetivos

Unir tudo que foi aprendido nas 9 aulas anteriores — MVC, banco de dados, ORM, async/await, EJS, REST API e integração front-back — e completar as duas peças que faltam para o sistema ficar pronto para avaliação: Autenticação (Login) e Empréstimos, além do Dashboard final.

Revisão do que já construímos

01	02	03
Ambiente Node.js configurado e projeto modularizado (Aula 01).	Servidor Express com rotas e middlewares (Aula 02).	Arquitetura MVC — controllers, routes, models (Aula 03).
04	05	06
Banco de dados MySQL com SQL puro (Aula 04).	ORM Sequelize — Models de Livro e Usuário (Aula 05).	Async/await e tratamento de erros em toda a aplicação (Aula 06).
07	08	09
Painel Web com EJS — CRUD completo de livros (Aula 07).	API REST completa e testável (Aula 08).	Frontend consumindo a API com Axios (Aula 09).

Parte 1 — Model de Empréstimos e Relacionamentos

Vamos criar a entidade que faltava: o Empréstimo, que relaciona um Usuário a um Livro.

models/Emprestimo.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/sequelize');

const Emprestimo = sequelize.define('Emprestimo', {
  dataEmprestimo: { type: DataTypes.DATE, defaultValue: DataTypes.NOW },
  dataDevolucaoPrevista: { type: DataTypes.DATE, allowNull: false },
  dataDevolucaoReal: { type: DataTypes.DATE, allowNull: true },
  status: {
    type: DataTypes.ENUM('em_andamento', 'devolvido', 'atrasado'),
    defaultValue: 'em_andamento',
  },
}, { tableName: 'emprestimos', timestamps: true });

module.exports = Emprestimo;
```

Agora criamos os relacionamentos entre os Models, em um arquivo central que é carregado antes do sync():

models/index.js

```
const Livro = require('./Livro');
const Usuario = require('./Usuario');
const Emprestimo = require('./Emprestimo');

// Um usuário tem vários empréstimos, um empréstimo pertence a um usuário
Usuario.hasMany(Emprestimo, { foreignKey: 'usuarioid' });
Emprestimo.belongsTo(Usuario, { foreignKey: 'usuarioid' });

// Um livro tem vários empréstimos, um empréstimo pertence a um livro
Livro.hasMany(Emprestimo, { foreignKey: 'livroid' });
Emprestimo.belongsTo(Livro, { foreignKey: 'livroid' });

module.exports = { Livro, Usuario, Emprestimo };
```



Parte 2 — Autenticação (Login)

Vamos proteger o painel com login, usando bcrypt para armazenar senhas de forma segura (hash, nunca texto puro) e JWT (JSON Web Token) para manter o usuário autenticado entre requisições.

```
npm install bcryptjs jsonwebtoken
```

Passo 1 — Criptografando a senha ao cadastrar

controllers/authController.js

```
const bcrypt = require('bcryptjs');
const Usuario = require('../models/Usuario');

async function cadastrar(req, res) {
  try {
    const { nome, email, senha } = req.body;
    const senhaCriptografada = await bcrypt.hash(senha, 10);
    const usuario = await Usuario.create({ nome, email, senha: senhaCriptografada });
    res.status(201).json({ id: usuario.id, nome: usuario.nome, email: usuario.email });
  } catch (erro) {
    res.status(400).json({ erro: erro.message });
  }
}
```

Passo 2 — Login e geração do Token JWT

controllers/authController.js (continuação)

```
const jwt = require('jsonwebtoken');
const SEGREDO_JWT = process.env.JWT_SECRET || 'segredo-super-secreto-trocar-em-producao';

async function login(req, res) {
  try {
    const { email, senha } = req.body;
    const usuario = await Usuario.findOne({ where: { email } });
    if (!usuario) { return res.status(401).json({ erro: 'E-mail ou senha inválidos' }); }
    const senhaValida = await bcrypt.compare(senha, usuario.senha);
    if (!senhaValida) { return res.status(401).json({ erro: 'E-mail ou senha inválidos' }); }
    const token = jwt.sign(
      { id: usuario.id, nome: usuario.nome, tipo: usuario.tipo },
      SEGREDO_JWT,
      { expiresIn: '8h' }
    );
    res.json({ token, usuario: { id: usuario.id, nome: usuario.nome, tipo: usuario.tipo } });
  } catch (erro) {
    res.status(500).json({ erro: erro.message });
  }
}

module.exports = { cadastrar, login };
```

Passo 3 — Middleware de Autenticação

Esse middleware protege rotas que exigem login — ele verifica se um token JWT válido foi enviado no header Authorization.

middlewares/autenticar.js

```
const jwt = require('jsonwebtoken');
const SEGredo_JWT = process.env.JWT_SECRET || 'segredo-super-secreto-trocar-em-producao';

function autenticar(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader) { return res.status(401).json({ erro: 'Token não fornecido' }); }
  const [, token] = authHeader.split(' '); // formato: "Bearer <token>"
  try {
    const dados = jwt.verify(token, SEGredo_JWT);
    req.usuario = dados; // disponibiliza os dados do usuário logado nas próximas funções
    next();
  } catch (erro) {
    return res.status(401).json({ erro: 'Token inválido ou expirado' });
  }
}

module.exports = autenticar;
```

Aplicando o middleware nas rotas protegidas:

routes/api/livroApi.js (trecho protegido)

```
const autenticar = require('../middlewares/autenticar');

router.post('/', autenticar, livroController.store);
router.put('/:id', autenticar, livroController.update);
router.delete('/:id', autenticar, livroController.destroy);

// GET continua público, qualquer um pode consultar os livros
```



Parte 3 — Empréstimos e Pesquisa

Parte 3 — Empréstimos (regra de negócio central)

controllers/emprestimoController.js

```
const { Emprestimo, Livro } = require('../models');

async function registrarEmprestimo(req, res) {
  try {
    const { livroid, dataDevolucaoPrevista } = req.body;
    const usuarioid = req.usuario.id; // vem do token JWT (middleware autenticar)
    const livro = await Livro.findByPk(livroid);
    if (!livro) return res.status(404).json({ erro: 'Livro não encontrado' });
    if (!livro.disponivel) return res.status(400).json({ erro: 'Livro indisponível para empréstimo' });
    const emprestimo = await Emprestimo.create({ livroid, usuarioid, dataDevolucaoPrevista });
    await livro.update({ disponivel: false });
    res.status(201).json(emprestimo);
  } catch (erro) { res.status(400).json({ erro: erro.message }); }
}

async function devolverLivro(req, res) {
  try {
    const emprestimo = await Emprestimo.findByPk(req.params.id);
    if (!emprestimo) return res.status(404).json({ erro: 'Empréstimo não encontrado' });
    await emprestimo.update({ status: 'devolvido', dataDevolucaoReal: new Date() });
    const livro = await Livro.findByPk(emprestimo.livroid);
    await livro.update({ disponivel: true });
    res.json(emprestimo);
  } catch (erro) { res.status(400).json({ erro: erro.message }); }
}

module.exports = { registrarEmprestimo, devolverLivro };
```

Parte 4 — Pesquisa de Livros

controllers/pesquisaController.js

```
const { Op } = require('sequelize');
const Livro = require('../models/Livro');

async function pesquisar(req, res) {
  try {
    const { termo } = req.query; // ex: /livros/pesquisa?termo=machado
    const livros = await Livro.findAll({
      where: {
        [Op.or]: [
          { titulo: { [Op.like]: `%${termo}%` } },
          { autor: { [Op.like]: `%${termo}%` } },
        ],
      },
    });
    res.json(livros);
  } catch (erro) { res.status(500).json({ erro: erro.message }); }
}

module.exports = { pesquisar };
```

Parte 5 — Dashboard

controllers/dashboardController.js

```
const { Livro, Usuario, Emprestimo } = require('../models');

async function dashboard(req, res) {
  try {
    const [totalLivros, totalUsuarios, emprestimosAtivos, livrosDisponiveis] = await Promise.all([
      Livro.count(),
      Usuario.count(),
      Emprestimo.count({ where: { status: 'em_andamento' } }),
      Livro.count({ where: { disponivel: true } }),
    ]);
    res.render('dashboard', {
      totalLivros,
      totalUsuarios,
      emprestimosAtivos,
      livrosDisponiveis,
    });
  } catch (erro) {
    res.status(500).send('Erro ao carregar dashboard');
  }
}

module.exports = { dashboard };
```

Repare que usamos `Promise.all()` (visto na Aula 06) para buscar as 4 métricas em paralelo, tornando o carregamento do dashboard muito mais rápido do que se cada consulta esperasse a anterior terminar.



Estrutura Final do Projeto

```
biblioteca-node/  
├── config/  
│   └── sequelize.js  
├── controllers/  
│   ├── authController.js  
│   ├── livroController.js  
│   ├── usuarioController.js  
│   ├── emprestimoController.js  
│   ├── pesquisaController.js  
│   └── dashboardController.js  
├── middlewares/  
│   └── autenticar.js  
├── models/  
│   ├── index.js  
│   ├── Livro.js  
│   ├── Usuario.js  
│   └── Emprestimo.js  
├── routes/  
│   ├── api/  
│   │   ├── livroApi.js  
│   │   ├── usuarioApi.js  
│   │   └── authApi.js  
│   ├── livroRoutes.js  
│   └── dashboardRoutes.js  
├── views/  
│   ├── partials/  
│   ├── livros/  
│   └── dashboard.ejs  
├── public/  
│   ├── css/  
│   └── js/  
├── .env  
├── server.js  
└── package.json
```



Deploy (opcional)

Para publicar o projeto e apresentá-lo online, algumas opções simples e gratuitas para começar:

- Backend: Render.com ou Railway.app (suportam Node.js + MySQL/PostgreSQL gratuitamente em planos iniciais).
- Banco de dados: um banco MySQL gerenciado no próprio Railway, ou um serviço como o Clever Cloud / FreeSQLDatabase para testes.
- Variáveis de ambiente: configure DB_HOST, DB_USER, DB_PASSWORD, JWT_SECRET direto no painel do serviço de deploy — nunca suba o arquivo .env para o GitHub (adicione-o ao .gitignore).

Avaliação Final

Cada aluno apresenta o projeto funcionando, demonstrando ao vivo:

- Cadastro de usuários e login funcionando com JWT.
- Cadastro de livros (via painel EJS e via API).
- Registro de um empréstimo e sua devolução, com atualização automática da disponibilidade do livro.
- Dashboard exibindo métricas reais do banco de dados.
- Funcionalidade de pesquisa de livros por título/autor.

Itens avaliados

Organização do código

nomes claros, sem lógica de negócio nas rotas.

Aderência à estrutura MVC.

Modelagem e uso correto do banco de dados

relacionamentos, tipos de dados adequados.

Cobertura de rotas

painel completo + API REST completa.

Qualidade da API

status HTTP corretos, tratamento de erros.

Usabilidade da interface

painel EJS navegável e funcional.

Funcionamento geral de ponta a ponta

sem erros ao demonstrar o fluxo completo.

✔️ 🎓 Parabéns! Ao concluir este projeto você terá construído, do zero, um sistema completo de nível profissional: Backend em Node.js/Express, arquitetura MVC, banco de dados relacional com ORM, autenticação segura com JWT, API REST documentada e testável, e um painel web funcional em EJS. Esse é exatamente o tipo de projeto que compõe um portfólio forte para o mercado de trabalho.